# Experiment – 1.2.1

**Student Name: Yatharth Bhaskar** **UID: 24IBD70014**

**Branch: AIT-CSE (Data Science)** **Section/Group: 24BDS-4 NTPP (A)**

**Semester: 5th** **Date of Performance: 1 5 July 2026**

**Subject: Full Stack-II** **Subject Code: 24CSP-337**

## 1. Aim:

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data

## 2. Software Required:

- Node.js & npm
- React.js
- Redux Toolkit
- React-Redux
- Code Editor (VS Code)

## 3. Objective:

- To understand the concept of global state management
- To implement Redux Toolkit for scalable state handling
- To design a normalized state structure
- To manage asynchronous data flows (optionally via mock APIs)

## 4. Theory:

As frontend applications grow in complexity, managing state across multiple components becomes increasingly challenging. Traditional approaches such as prop drilling lead to tightly coupled components and reduced maintainability.

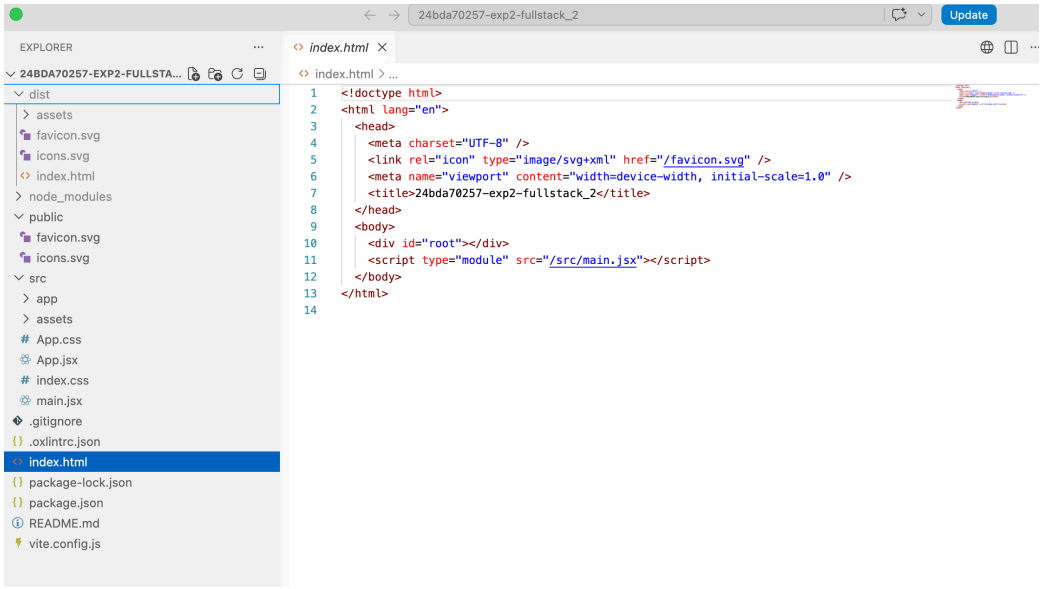
Redux provides a centralized store that acts as a **single source of truth**, enabling predictable state transitions through actions and reducers. Redux Toolkit simplifies Redux implementation by reducing boilerplate and providing built-in utilities for common tasks.

State normalization is an important concept where data is stored in a structured format (similar to relational databases), reducing redundancy and improving performance. This is especially useful when managing collections such as posts or users.Additionally, Redux supports handling asynchronous operations, enabling integration with APIs or simulated mock data sources.

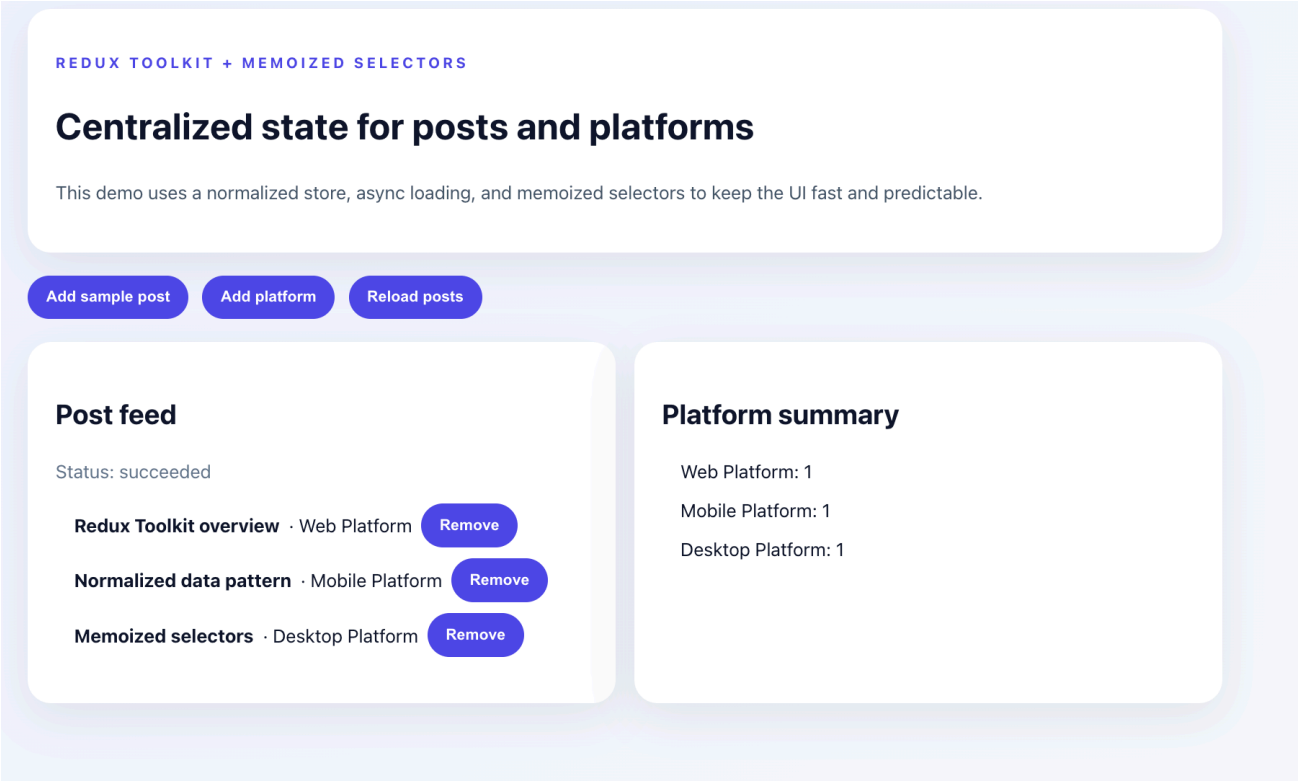
## 5. Procedure/Algorithm:

- Install Redux Toolkit and React-Redux
- Configure Redux store
- Create slices for posts and platforms
- Define initial state structure
- Implement reducers for CRUD operations
- Connect components to Redux store using hooks
- *(Optional)* Integrate mock API calls using async thunks

## 6. Code:



## 7. Output:


REDUX TOOLKIT + MEMOIZED SELECTORS
Centralized state for posts and platforms
This demo uses a normalized store, async loading, and memoized selectors to keep the UI fast and predictable.
Add sample post
Add platform
Reload posts
Post feed
Status: succeeded
Redux Toolkit overview · Web Platform
Remove
Normalized data pattern · Mobile Platform
Remove
Memoized selectors · Desktop Platform
Remove
Platform summary
Web Platform: 1
Mobile Platform: 1
Desktop Platform: 1

## 8. Conclusion:

- Centralized state management system implemented
- Efficient handling of posts and platform data
- Reduced prop drilling
- Scalable and maintainable state architecture

## 9. Learning Outcomes:

- Understand the need for centralized state management in React applications.
- Learn how Redux stores and manages application state.
- Implement actions and reducers using Redux Toolkit.
- Manage shared state efficiently across multiple components.
- Build scalable and maintainable frontend applications using Redux.